INFORME DE ACTIVIDAD 6

Taller Práctico de Repaso y Consolidación en C++

Programa Integrador Anual (P.I.A.) — E.E.S.T. N.º 1 "Eduardo Ader"

Datos del estudiante

Estudiante: Malek Cheheid
Curso: 5° Año — 3° "A"
Colegio: E.E.S.T. N.º 1 "Eduardo Ader", Vicente López
Materia: Laboratorio de Programación (LPR)
Profesor: Yamil Ganduglia
Actividad: N.º 6 — Taller de Repaso e Integración
Entrega: 01 de Octubre de 2026

Repositorio del proyecto:

https://github.com/malekcheheid-art/repasogeneral.git

(Enlace directo al repositorio de GitHub del trabajo)

Versión del entregable: v1.0.0 — Baseline Aprobado
DocVer / SemVer

1. Introducción

Este informe documenta la Actividad 6 del taller de Laboratorio de Programación. El objetivo es consolidar tres pilares del bajo nivel en C++: la recursividad con caso base, la búsqueda secuencial sobre memoria contigua y la modificación directa de variables mediante punteros.

El trabajo se organiza en un único programa (src/main.cpp) que ejecuta los tres retos en forma secuencial y muestra en pantalla los resultados, incluyendo direcciones de memoria en formato hexadecimal.

El código fuente, las evidencias de ejecución y este informe se mantienen en el repositorio indicado en la portada.

2. Metodología de aprendizaje 5R

• R1 – Registrar: compilar y ejecutar el programa en PowerShell (VS Code) para ver la salida real y las direcciones hexadecimales.

• R2 – Reducir: identificar el 20 % del código que produce el 80 % del resultado (caso base, break y variable temporal).

• R3 – Responder: justificar por qué el programa se comporta así a bajo nivel (sección 5).

• R4 – Reflexionar: relacionar la lógica con el Proyecto Integrador Anual (lectura de sensores, buffers).

• R5 – Revisar: confirmar que no haya errores de compilación, hacer git push a GitHub y GitLab y actualizar docs/CHANGELOG.md.

3. Desarrollo de los retos

Reto 1 — Suma recursiva de 1 hasta n

La función se invoca a sí misma disminuyendo n. El caso base (n <= 0) corta la cadena de llamadas y evita el desbordamiento de pila (Stack Overflow).

```
int sumaRecursiva(int n) {
    if (n <= 0) return 0;   // caso base
    return n + sumaRecursiva(n - 1);
}
```

Entrada usada: n = 5. Salida esperada: 1 + 2 + 3 + 4 + 5 = 15.

Reto 2 — Búsqueda secuencial en memoria contigua

Se recorre un arreglo de 10 enteros celda por celda. Al encontrar el valor 18 se imprime su índice y su dirección en RAM, y se interrumpe el ciclo con break para optimizar el tiempo de ejecución (O(n) en el peor caso, menos en promedio).

```
int vectorDatos[10] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
int numeroBuscar = 18;
// ... for (int i = 0; i < TAM; i++) { if (...) { ... break; } }
```

Salida obtenida: valor 18 hallado en el índice [6], con dirección hexadecimal de la celda.

Reto 3 — Intercambio de valores mediante punteros

Se pasan las direcciones de x e y a la función. Con la desreferenciación (*) se modifica el valor guardado en esas celdas de memoria, intercambiando físicamente los valores originales.

```
void intercambiarValores(int* ptrA, int* ptrB) {
    int temporal = *ptrA;
    *ptrA = *ptrB;
    *ptrB = temporal;
}
```

Entrada: x = 100, y = 500. Salida: x = 500, y = 100.

4. Evidencias de compilación y ejecución

El programa se compiló con g++ (TDM-GCC 10.3.0) y se ejecutó en Windows PowerShell. La captura de la terminal muestra la salida completa de los tres retos.

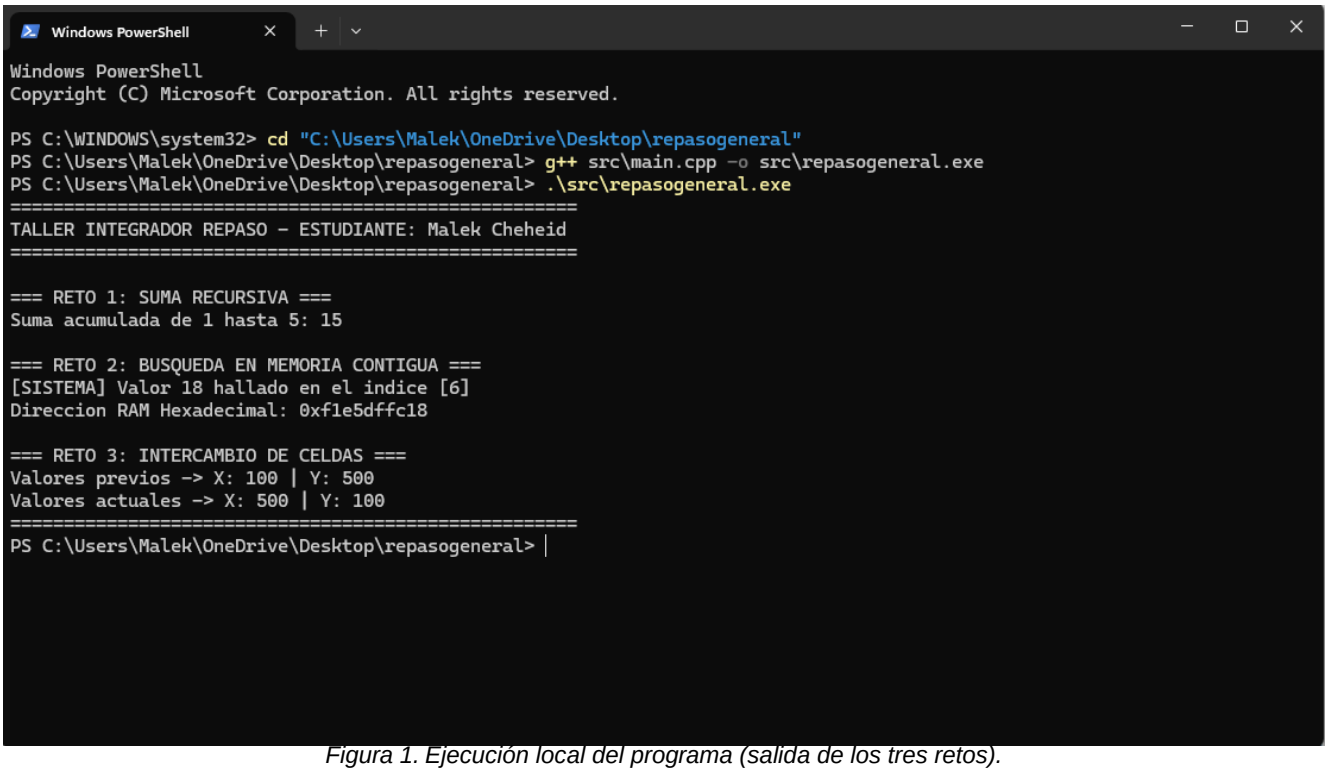


Figura 1. Ejecución local del programa (salida de los tres retos).

La traza de memoria muestra el bloque contiguo del arreglo, la celda hallada con su dirección y el intercambio físico de x e y.

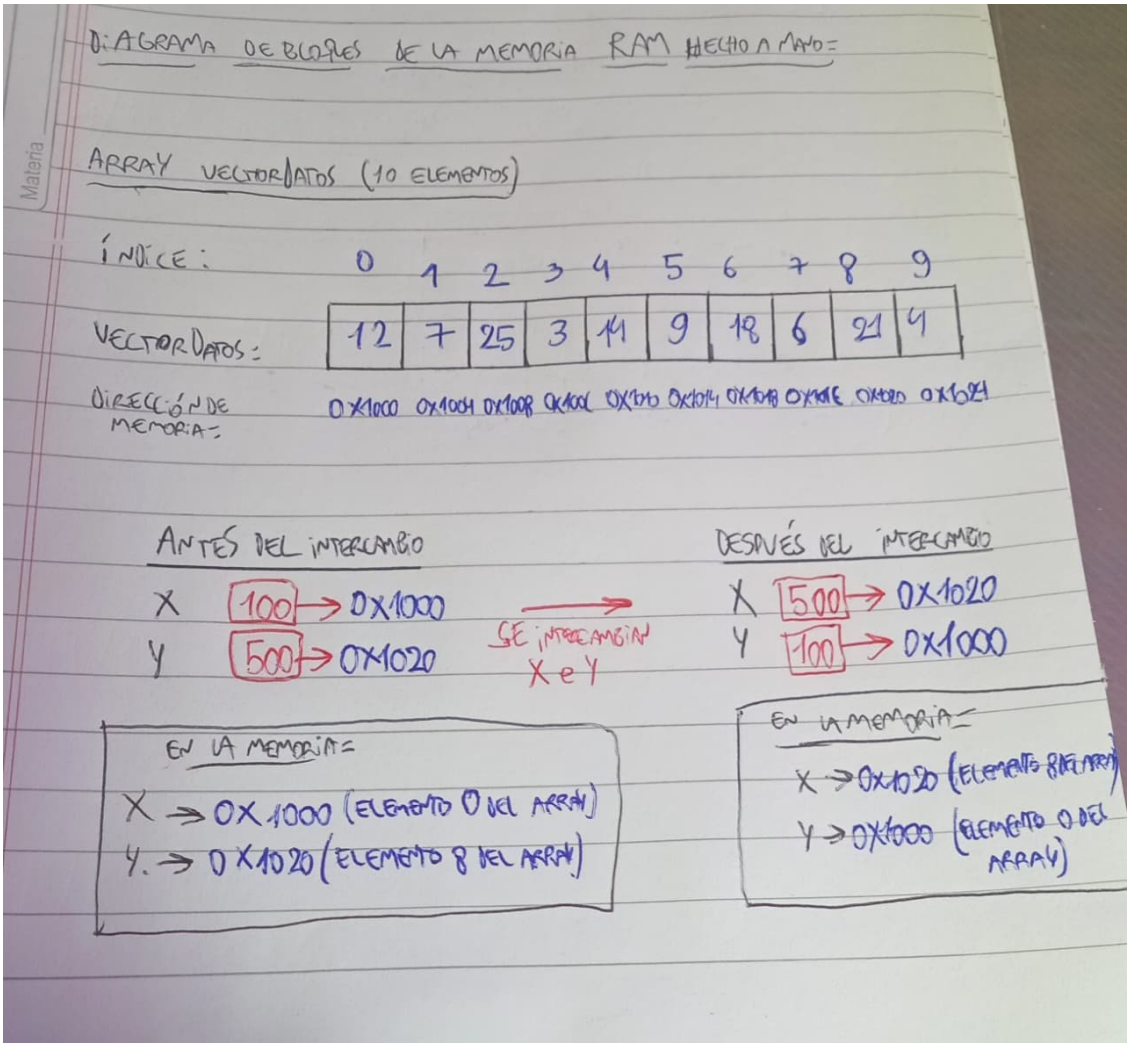


Figura 2. Diagrama de bloques de la memoria RAM (hecho a mano).

Configuración de control de versiones (dual-remote) verificada con git remote -v: un destino de fetch (GitHub) y dos destinos de push (GitHub y GitLab).

```
origin  https://github.com/malekcheheid-art/repasogeneral.git (fetch)
origin  https://github.com/malekcheheid-art/repasogeneral.git (push)
origin  https://gitlab.com/malekcheheid-art/repasogeneral.git (push)
```

5. Cuestionario técnico de control

Las respuestas son personales y técnico-justificadas según lo solicitado por la consigna.

1. Caso Base: ¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?

Sin caso base, la función se llama a sí misma sin parar. Cada llamada apila un marco (parámetros, variables locales y dirección de retorno) en la pila (stack). Como el stack tiene tamaño finito, en algún punto la memoria reservada se agota y ocurre un Stack Overflow, que normalmente finaliza el programa de forma abrupta.

2. Eficiencia: ¿Por qué la instrucción break optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?

break termina el ciclo en cuanto se encuentra el elemento buscado, evitando comparaciones innecesarias con las celdas restantes. Reduce la cantidad de instrucciones ejecutadas y el número de accesos a memoria, mejorando el tiempo promedio aunque el peor caso siga siendo O(n).

3. Direccionamiento: ¿Cómo comprobar mediante el operador & que los elementos de un array ocupan celdas contiguas en la RAM?

Imprimiendo &vectorDatos[i] para cada índice se observa que las direcciones aumentan de forma regular (normalmente de 4 bytes por int en x86/x64), lo que demuestra que el arreglo se almacena en posiciones consecutivas de memoria, como define un bloque contiguo.

4. Variables Temporales: ¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?

La variable temporal guarda el valor original de la primera celda (*ptrA) antes de que sea sobreescrito. Así no se pierde ese dato y es posible completar la rotación de valores, consiguiendo el intercambio físico de x e y.

5. Autoevaluación: ¿Cuál de los 3 retos te resultó más complejo de comprender a bajo nivel y de qué manera lograste resolverlo?

El reto de punteros fue el más complejo por la diferencia entre la variable y su dirección. Lo resolví dibujando un diagrama de memoria con bloques y flechas (ptrA, ptrB, temporal) y verificando cada paso compilando y mostrando las direcciones por pantalla.

6. Tabla de Control de Revisiones (DocVer)

```
Versión  | Fecha      | Estado            | Descripción
v1.0.0   | 2026-10-15 | Baseline aprobado  | Estructura inicial del repo, 3 retos
unificados,
         |            |                   | capturas de ejecución y de memoria RAM,
informe APA v7.
```

7. Conclusión

El taller permitió afianzar el manejo de la pila, la memoria contigua de los arreglos y el pasaje por dirección mediante punteros. Comprender estas bases es útil para futuras tareas del Proyecto Integrador Anual, como la gestión de buffers de datos de sensores en tiempo real.

El repositorio quedó versionado con la estructura solicitada y sincronizado en GitHub y GitLab mediante configuración dual-remote.

8. Referencias técnicas

- Recursión en C++ — Conceptos de Recursividad (GeeksforGeeks).
- GeeksforGeeks — Búsqueda Secuencial (Linear Search).
- Tutorial de C++ — Punteros y Dirección de Memoria (TutorialsPoint).
- Git Documentation — Configuración de Git Remote y Dual Push.